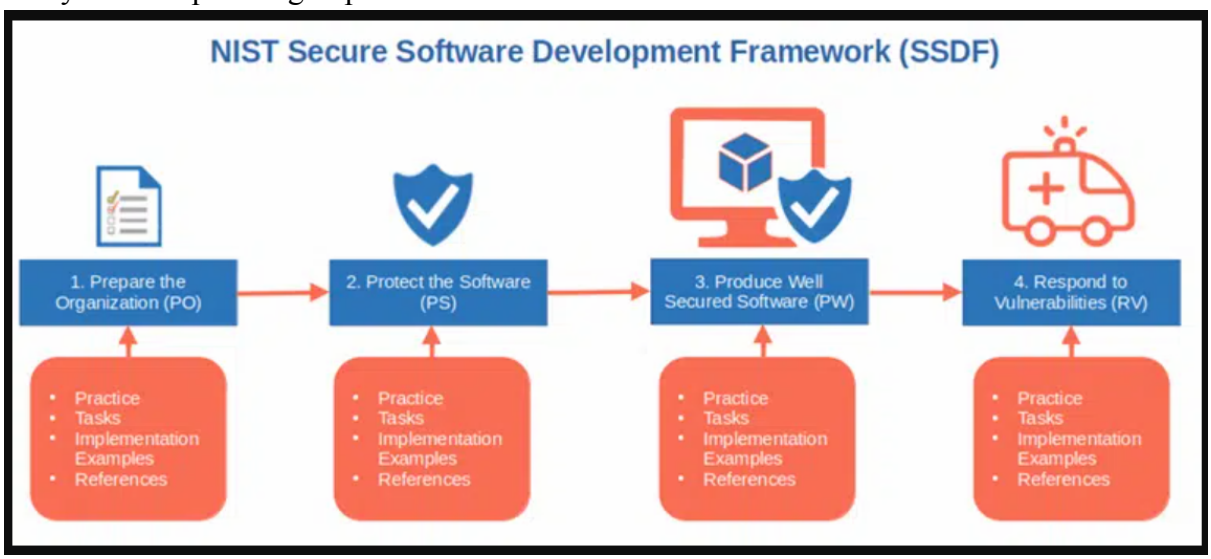
The **Secure Software Development Framework (SSDF)** is a set of high-level, outcome-based best practices published by the U.S. National Institute of Standards and Technology (NIST) as **SP 800-218**. Its primary goal is to help organizations seamlessly integrate security into every stage of the software development lifecycle (SDLC) to reduce vulnerabilities and mitigate risks.

The Four Core Practice Groups of SSDF

The SSDF organizes its 42 recommended practices into four key groups that cover the entire lifecycle from planning to post-release.



Practice Group	Primary Goal	Key Activities
Prepare the Organization (PO)	Ensure the organization has the people, processes, and technology to perform secure development.	Defining security requirements, assigning roles, implementing secure toolchains, and maintaining secure development environments.
Protect the Software (PS)	Safeguard all software components (e.g., source code, executables) from tampering and unauthorized access.	Restricting access based on least privilege, using code signing, and protecting the integrity of the software supply chain.
Produce Well-Secured Software (PW)	Minimize security vulnerabilities in releases.	Reviewing code, using static/dynamic analysis tools, managing third-party components, and securing the build process.
Respond to Vulnerabilities (RV)	Identify and remediate vulnerabilities post-release.	Receiving and triaging reports, patching, and communicating fixes to end-users.

What's New in Proposed Version 1.2

The latest draft of the framework expands its focus from just *developing* secure code to *operating* it securely. The most significant proposed change is a new practice, **PO.6: "Define and Implement a Continuous Process Improvement Plan"**.

This change reflects a shift from point-in-time compliance (e.g., passing an audit) to **continuous validation**, which includes:

- **Proactive Vulnerability Hunting**: Actively searching for errors rather than waiting for them to be reported, as fixing issues post-launch can be up to **100 times more expensive** than addressing them during design.
- **Continuous Environment Updates**: Regularly updating development environments (IDEs, operating systems, tooling) to keep pace with emerging threats.
- **Measuring Escape Rate**: Tracking how many vulnerabilities are discovered *after* the software is released to gauge the effectiveness of internal security controls.

How the SSDF is Used in the Real World

While the SSDF itself is guidance, it has become the baseline for many regulations and industry standards. You might encounter it through:

- **U.S. Government Requirements**: For software sold to the U.S. government, adherence to SSDF practices is effectively mandatory under OMB Memorandum M-22-18.
- **International Standards**: The framework is being adopted globally. ETSI, a European standards body, is developing a standard (TS 104 219) specifically for implementing SSDF. It is also a key reference for aligning with the EU's Cyber Resilience Act (CRA).
- **Industry & Commercial Compliance**: It is widely adopted as a best practice for supply chain security, with vendors providing "Secure by Design" guides and mapping their tools (e.g., GitHub, SCA scanners) to SSDF tasks to help customers demonstrate compliance.

The framework is flexible and designed to work with any development model, from Waterfall to Agile and DevSecOps, by focusing on desired *outcomes* rather than prescribing specific tools.

OWASP's application security verification standard

The **OWASP Application Security Verification Standard (ASVS)** is a comprehensive framework and technical checklist designed to help organizations verify the security of their web applications and APIs. If the **NIST SSDF** answers "what" secure development practices an organization needs, the **OWASP ASVS** provides the detailed, testable "how" for the actual software.

The **OWASP Application Security Verification Standard (ASVS)** is a comprehensive framework and technical checklist designed to help organizations verify the security of their web applications and APIs. If the **NIST SSDF** answers "what" secure development practices an organization needs, the **OWASP ASVS** provides the detailed, testable "how" for the actual software.

Core Purpose: A Verifiable Standard for AppSec

Unlike high-level frameworks, the ASVS is primarily a **verification standard**. It provides a basis for:

- **Developers & Architects:** As a detailed guide for secure design and implementation.
- **Security Testers:** As a checklist for penetration testing and code review scope.
- **Organizations:** To procure secure software by setting clear, technical requirements.

The Three Verification Assurance Levels (L1, L2, L3)

The ASVS recognizes that not all applications face the same risk. It organizes its security requirements into three distinct levels, allowing you to scale your efforts based on the application's sensitivity.

Level	Target Application	What It Means
Level 1 (L1)	Low-risk applications, static informational sites.	**Penetration testable**. All security controls must be in place and effective, but they are largely automated and opportunistic. It is the minimum level for any application.
Level 2 (L2)	Most business applications handle sensitive data (e.g., PII, payroll, and financial).	**The recommended baseline**. Requires defense-in-depth controls and is the standard for most software, requiring both automated tooling and manual verification.
Level 3 (L3)	Highly sensitive applications (e.g., military, healthcare, critical financial systems).	**The highest assurance**. Requires in-depth code reviews, verification of advanced architectural controls, and rigorous testing of every security mechanism.

The 14 Key Control Areas (V1 to V14)

The framework is divided into 14 chapters (V1-V14), each covering a critical security domain. The latest version (5.0) is organized better to reflect modern architecture, including APIs and cloud-native components.

Category	Focus Area	Example Control
V1	**Architecture, Design & Threat Modeling**	Ensuring a formal threat model exists for the application.
V2	**Authentication**	Implementing Multi-Factor Authentication (MFA) and preventing brute-force attacks.

V3	**Session Management**	Secure generation, storage, and invalidation of session tokens.
V4	**Access Control**	Enforcing the least privilege principle and preventing IDOR (Insecure Direct Object References).
V5	**Validation, Sanitization & Encoding**	Preventing injection flaws (SQL, NoSQL, OS command) via input validation.
V6	**Stored Cryptography**	Proper management of keys and algorithms for data at rest.
V7	**Error Handling & Logging**	Ensuring errors do not leak stack traces and that suspicious activities are logged.
V8	**Data Protection**	Protecting data in transit (TLS) and specific sensitive data in memory.
V9	**Communications**	Secure configuration of HTTP headers (CSP, CORS) and TLS settings.
V10	**Malicious Code**	Verifying the integrity of the codebase against backdoors or logic bombs.
V11	**Business Logic**	Validating that workflows cannot be subverted to perform unintended actions.
V12	**Files & Resources**	Secure file upload handling (type validation, malware scanning).
V13	**API & Web Service**	Specific controls for REST and GraphQL (e.g., mass assignment, rate limiting).
V14	**Configuration**	Hardening the runtime environment and removing unnecessary features.

Practical Use Cases in the SDLC

You don't just "read" the ASVS; you operationalize it across the development lifecycle.

- **For Secure Requirements:** Instead of vague instructions ("make it secure"), reference specific ASVS requirements (e.g., "must meet ASVS L2 V2.7 for OAuth 2.0").
- **For Testing Scoping:** Penetration testers use the ASVS to map their findings to standard IDs, ensuring broad coverage.
- **For CI/CD Automation:** Integrate scanners (SAST/DAST) that flag violations of specific ASVS controls directly in pull requests.

How ASVS Relates to NIST SSDF & Other Standards

The ASVS does not exist in a vacuum. It complements the NIST SSDF by providing technical details.

- **Mapping to NIST SSDF:** The **SSDF** tells you *to* "Produce Well-Secured Software" (Practice **PW**). The **ASVS** tells you *exactly* how to verify that the "Authentication" (V2) or "Access Control" (V4) works.
- **OWASP Top 10:** The Top 10 identifies *risks* (e.g., Broken Access Control). The ASVS provides the *controls* to fix and test them.
- **SAMM & OpenCRE:** Frameworks like OWASP SAMM use **OpenCRE** (Common Requirement Enumeration) to link their governance activities directly to specific ASVS technical requirements.

How to Get Started

1. **Pick Your Level:** Decide if your app needs L1, L2, or L3 based on its risk profile.
2. **Map to Your Pipeline:** Start by implementing automated checks for the "low-hanging fruit" (like secure headers or injection flaws) in your CI/CD pipeline.
3. **Use the Checklist:** Download the official ASVS checklist to guide your code reviews and testing cycles.

OpenSAMM & BSIMM – Maturity Models for Software Security

From Verification to Maturity

The OWASP ASVS is a verification standard that tells you *how* to test and verify the security of a specific application. It answers questions like, "Does this API have proper access controls?" But what about the bigger picture? How do you measure and improve the *entire* software security program across your *whole organization*? That's where maturity models come in.

There are two leading frameworks: OpenSAMM (the OWASP Software Assurance Maturity Model) and the BSIMM (Building Security In Maturity Model). While both help organizations enhance their application security, they take fundamentally different approaches. Understanding this difference is key to using them effectively.

The Core Difference: Prescriptive vs. Descriptive

The single most important distinction between OpenSAMM and BSIMM is their philosophy.

- OpenSAMM is *prescriptive*. It tells you *what you should do*. Think of it as a roadmap or a guidebook. It provides a structured set of recommended security practices, organized by business function, and allows you to define a target maturity level (0-3) for each. If you're asking, "We're a fintech startup just getting started with security; what should our roadmap look like?", SAMM is designed to give you an answer. It is an open-source framework, created by Pravir Chandra in 2009 and now maintained by OWASP.
- BSIMM is *descriptive*. It tells you *what others are actually doing*. It is not a "how-to" guide. Instead, it is a measurement tool and a data set. The BSIMM team studies the software security initiatives of dozens of participating organizations—such as Cisco, HSBC, Adobe, and many others—and publishes an anonymized summary of the activities they observe. If you're asking, "How does our security program compare to our peers in the financial services industry?", BSIMM provides the benchmark data.

To put it simply: SAMM is the *prescribed* diet and exercise plan for getting fit. BSIMM is a *survey* of what 100+ companies are *actually* eating and doing at the gym.

OpenSAMM: The Prescriptive Roadmap

OpenSAMM provides a flexible, measurable way to build and improve a software security program. It is built around five core business functions that span the entire software development lifecycle :

1. Governance: Cross-cutting activities like strategy, metrics, policy, and training.
2. Design: Activities related to threat modeling, security requirements, and architecture analysis.
3. Implementation: Secure coding standards, secret management, and defect management.
4. Verification: Security testing, including code review, penetration testing, and vulnerability management.
5. Operations: Incident management, environment management, and operational monitoring.

Within these five functions, SAMM defines 15 security practices (e.g., "Education & Guidance," "Threat Assessment," "Secret Management"). Each practice has three maturity levels (Level 1 = Initial, Level 2 = Structured, Level 3 = Optimized). You don't have to be Level 3 in everything. The power of SAMM is that you can assess your current state, define a target state based on your specific business risks, and then execute an incremental roadmap to improve one practice at a time.

Key strengths of SAMM: It is risk-driven, business-focused, and designed to be iterative. It's excellent for organizations that want a clear, structured plan to build their AppSec program from scratch or to formalize ad hoc efforts.

BSIMM in Detail: The Descriptive Benchmark

BSIMM is fundamentally a data-driven study. Since 2009, it has collected data from over 120 participating organizations, observing over 300,000 developers and thousands of Software Security Group (SSG) members. The output is the annual BSIMM report, which categorizes observed activities into 12 practices across four domains:

- Governance: Strategy & Metrics, Compliance & Policy, Training.
- Intelligence: Attack Models, Security Features & Design, Standards & Requirements.
- SSDL Touchpoints: Architecture Analysis, Code Review, Security Testing.
- Deployment: Penetration Testing, Software Environment, Configuration Management & Vulnerability Management.

For each of these practices, BSIMM lists the specific activities that were observed at participating firms. For example, under "Code Review," an observed activity might be "Use automated tools to find security defects in all critical software." The model then shows what percentage of firms are doing that activity.

Key strengths of BSIMM: It provides an objective, real-world benchmark. You can use it to see how your program stacks up against your industry peers, identify gaps you hadn't considered, and justify investments to leadership with data. As the CMU podcast notes, having a "data-based measurement is unbelievably powerful" when making a business case to executives.

SAMM vs. BSIMM: A Practical Comparison & How They Work Together

So which one do you use? The answer is: they are complementary, not competing.

- Use OpenSAMM when: You need a *plan*. You are building a new program, or your existing program lacks structure. SAMM gives you the "should-do" roadmap tailored to your risk tolerance. It helps you answer the question, "What's our next step?"
- Use BSIMM when: You need a *checkup*. You have an established program, and you want to know how effective it is compared to the industry. BSIMM gives you the "are-we-doing-the-right-things" reality check. It helps you answer the question, "How do we compare to our peers?"

How they map to each other: A detailed mapping published by the SAMM core team in late 2024 shows significant overlap but also unique activities. For instance, BSIMM has an observed activity around "making SSI efforts part of external marketing" (using security as a selling point), which SAMM doesn't prescribe. Conversely, SAMM has a dedicated practice for "Secret Management," which BSIMM doesn't explicitly call out as a standalone activity, even though aspects of it are observed across other practices.

Conclusion & How They Fit with ASVS

To tie it all back to our previous lecture on the OWASP ASVS:

- BSIMM tells you *what the industry is doing* (descriptive benchmarking).
- SAMM tells you *what you should do next* (prescriptive roadmap).
- ASVS tells you *how to verify it's done correctly* (technical verification).

You could use BSIMM to identify a gap in your code review practices. You would then use SAMM to build a roadmap to improve that practice from Level 1 to Level 2. Finally, you would use the ASVS (specifically V5 for validation or controls for code review) as the technical checklist to ensure developers are actually implementing the controls correctly. Together, they form a powerful, layered approach to mature, measure, and verify software security.